Knockoff Filter

Michele Filopanti, Elena Maggiore, Marco Rosato

2025-11-22

Importing the data

Data is found at https://www.kaggle.com/datasets/shashanknecrothapa/ames-housing-dataset. We will mostly consider the numeric columns, as categorical columns are not suitable for the construction of knockoff variables. However, categorical columns will still prove useful in the preprocessing phase.

```
df0 <- read.csv("AmesHousing.csv") |> dplyr::select(-Order, -PID) |>
Filter(f= function(x)length(unique(x)) > 0)
colnames(df0) <- gsub("\\.", "",  colnames(df0))
df <- df0  |> select_if(is.numeric) |> select(-MSSubClass, -TotalBsmtSF)
summary(df)[, 1:10]
```

```
na.fill <- function(col) {
  is.num <- is.numeric(col)
  col[is.na(col)] <- ifelse(is.num, 0, "None")
  if(is.num) col else as.factor(col)
}
```

```
##   LotFrontage        LotArea        OverallQual      OverallCond
##  Min.   : 21.00   Min.   :  1300   Min.   : 1.000   Min.   :1.000
##  1st Qu.: 58.00   1st Qu.:  7440   1st Qu.: 5.000   1st Qu.:5.000
##  Median : 68.00   Median :  9436   Median : 6.000   Median :5.000
##  Mean   : 69.22   Mean   : 10148   Mean   : 6.095   Mean   :5.563
##  3rd Qu.: 80.00   3rd Qu.: 11555   3rd Qu.: 7.000   3rd Qu.:6.000
##  Max.   :313.00   Max.   :215245   Max.   :10.000   Max.   :9.000
##  NA's   :490
##    YearBuilt     YearRemodAdd    MasVnrArea       BsmtFinSF1
##  Min.   :1872   Min.   :1950   Min.   :   0.0   Min.   :   0.0
##  1st Qu.:1954   1st Qu.:1965   1st Qu.:   0.0   1st Qu.:   0.0
##  Median :1973   Median :1993   Median :   0.0   Median : 370.0
##  Mean   :1971   Mean   :1984   Mean   : 101.9   Mean   : 442.6
##  3rd Qu.:2001   3rd Qu.:2004   3rd Qu.: 164.0   3rd Qu.: 734.0
##  Max.   :2010   Max.   :2010   Max.   :1600.0   Max.   :5644.0
##                                NA's   :23       NA's   :1
##    BsmtFinSF2        BsmtUnfSF
##  Min.   :   0.00   Min.   :   0.0
##  1st Qu.:   0.00   1st Qu.: 219.0
##  Median :   0.00   Median : 466.0
##  Mean   :  49.72   Mean   : 559.3
##  3rd Qu.:   0.00   3rd Qu.: 802.0
##  Max.   :1526.00   Max.   :2336.0
##  NA's   :1         NA's   :1
```

It is immediately visible that some columns present missing values. Here are the columns containing missing values, with the respective count:

```
df |> sapply(function(x)sum(is.na(x))) |> Filter(f=identity) -> na.counts
print(na.counts)
```

```
##  LotFrontage   MasVnrArea   BsmtFinSF1   BsmtFinSF2    BsmtUnfSF
BsmtFullBath
##          490           23            1            1            1
2
## BsmtHalfBath  GarageYrBlt   GarageCars   GarageArea
##            2          159            1            1
```

The `LotFrontage` variable is missing in a sizeable portion of the records, and would therefore be unwise to remove. Same goes for `GarageYrBlt`. The other variables contain a minimal amount of missing values. most of them in the same row, meaning we can safely discard the incomplete records.

```
na.counts |> Filter(f = function(x)x < 100) |> names() -> cols.to.check
rows.to.keep <- complete.cases(df[, cols.to.check])
df <- df[rows.to.keep, ]
df0 <- df0[rows.to.keep, ]
```

If we inspect the values of the other garage-related variables when `GarageYrBlt` is missing, we can see that these missing values are not at random:

```
df[is.na(df$GarageYrBlt), c("GarageYrBlt", "GarageCars", "GarageArea")]
```

```
##      GarageYrBlt GarageCars GarageArea
## 28            NA          0          0
## 120           NA          0          0
## 126           NA          0          0
## 130           NA          0          0
## 131           NA          0          0
## 171           NA          0          0
## 172           NA          0          0
## 187           NA          0          0
## 204           NA          0          0
## 207           NA          0          0
…                 …          …          …
```

meaning that a missing `GarageYrBlt` value is not due to registration errors, but rather signifying a house with a missing garage. We will therefore fill these NA values with zeros.

```
df <- transform(df, GarageYrBlt = na.fill(GarageYrBlt))
```

Regarding `LotFrontage`, can notice one thing: there are some other lot-related variables in the dataset, those being `LotShape` (categorical), `LotConfig` (categorical) and `LotArea` (numeric). Together with `Neighbourhood` and `BldgType` (both categorical), they seem to explain decently well the missingness of `LotFrontage`.

We can see that houses whose `LotShape` is regular are much less likely to have a missing `LotFrontage`:

```
cols <- c("LotShape", "LotFrontage", "LotConfig", "LotArea", "Neighborhood",
"BldgType")
df.lot <- df0[cols]

df.lot |> with(table(is.na(LotFrontage), LotShape)) |> apply(2, function(x) x
/ sum(x))
```

```
##        LotShape
##               IR1       IR2    IR3        Reg
##   FALSE 0.6670114 0.6266667 0.6875 0.93011918
##   TRUE  0.3329886 0.3733333 0.3125 0.06988082
```

therefore, before modeling the missingness, we will group together all irregular lot shapes in a single level. We will also merge all `Neighborhood` levels which have 0 missing response values with the level with the least amount of missing values.

```
sapply(df.lot$LotShape, function(x)ifelse(x != "Reg", "Irreg", x)) ->
df.lot$LotShape

with(df.lot, table(is.na(LotFrontage), Neighborhood)) |> apply(2, function(x)
x / sum(x)) -> ngbr.table
ngbr.table
```

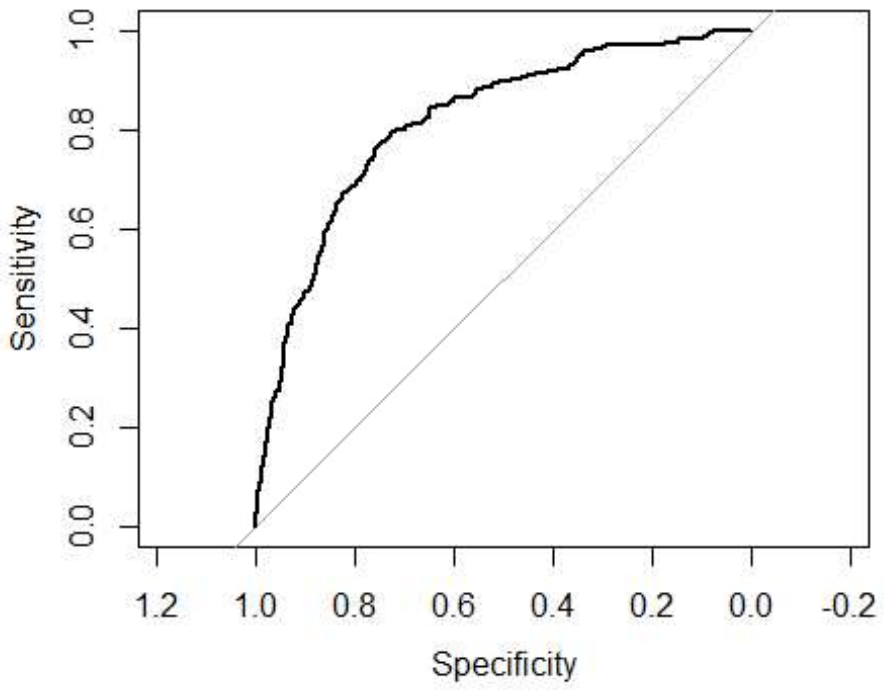
```
##        Neighborhood
##           Blmngtn Blueste BrDale   BrkSide   ClearCr   CollgCr   Crawfor
##   FALSE 0.7142857       1      1 0.8785047 0.4545455 0.8226415 0.8137255
##   TRUE  0.2857143       0      0 0.1214953 0.5454545 0.1773585 0.1862745
##        Neighborhood
##            Edwards Gilbert Greens GrnHill     IDOTRR Landmrk   MeadowV
##   FALSE 0.91666667  0.6875  0.875       0 0.93478261       0 0.8918919
##   TRUE  0.08333333  0.3125  0.125       1 0.06521739       1 0.1081081
##        Neighborhood
##           Mitchel     NAmes   NoRidge    NPkVill    NridgHt   NWAmes
OldTown
##   FALSE 0.7894737 0.8465011 0.7605634 0.91304348 0.98170732 0.648855
0.958159
##   TRUE  0.2105263 0.1534989 0.2394366 0.08695652 0.01829268 0.351145
0.041841
##        Neighborhood
##            Sawyer   SawyerW   Somerst    StoneBr      SWISU    Timber
Veenker
##   FALSE 0.6490066 0.8467742 0.8895349 0.90196078 0.91666667 0.7887324
```

```
0.6666667
##   TRUE  0.3509934 0.1532258 0.1104651 0.09803922 0.08333333 0.2112676
0.3333333
```

```
ngbr.table[1, ] |> sort(decreasing = T) |> names() -> to.fuse
to.fuse <- to.fuse[1:3]
df.lot$Neighborhood <- sapply(df.lot$Neighborhood, function(x)ifelse(x %in%
to.fuse, "Other", x)) |> as.factor() |> relevel(ref = "Other")
```

From the logistic model below…

```
df.lot$LotFrontage |> is.na() |> as.factor() -> df.lot$LotFrontage

lot.model <- glm(LotFrontage ~ ., data = df.lot, family = binomial())
lot.roc <- roc(df.lot$LotFrontage, fitted(lot.model))

## Setting levels: control = FALSE, case = TRUE

## Setting direction: controls < cases

plot.roc(lot.roc)
```




```
lot.roc$auc

## Area under the curve: 0.8147
```

… we can see that the selected variables appear to explain decently well the missingness of `LotFrontage`. By checking the coefficients effects of each variable, we can see that lots with a

regular shape indeed are much more likely to not have a missing lot frontage value. Also, inside lots and cul-de-sac lots are more prone to missingness.

```
summary(lot.model)

##
## Call:
## glm(formula = LotFrontage ~ ., family = binomial(), data = df.lot)
##
## Coefficients:
##                       Estimate Std. Error z value Pr(>|z|)
## (Intercept)         -3.645e+00  6.148e-01  -5.928 3.06e-09 ***
## LotShapeReg         -1.794e+00  1.353e-01 -13.258  < 2e-16 ***
## LotConfigCulDSac     4.047e-01  2.132e-01   1.898 0.057678 .
## LotConfigFR2         2.215e-02  3.197e-01   0.069 0.944754
## LotConfigFR3         4.590e-01  7.670e-01   0.598 0.549521
## LotConfigInside     -4.685e-01  1.446e-01  -3.241 0.001193 **
## LotArea              9.347e-06  6.507e-06   1.437 0.150847
## NeighborhoodBlmngtn  4.248e+00  7.446e-01   5.706 1.16e-08 ***
## NeighborhoodBrkSide  3.149e+00  6.808e-01   4.625 3.74e-06 ***
## NeighborhoodClearCr  4.529e+00  6.956e-01   6.511 7.48e-11 ***
## NeighborhoodCollgCr  2.909e+00  6.242e-01   4.661 3.15e-06 ***
## NeighborhoodCrawfor  2.834e+00  6.529e-01   4.341 1.42e-05 ***
## NeighborhoodEdwards  2.561e+00  6.610e-01   3.874 0.000107 ***
## NeighborhoodGilbert  3.268e+00  6.268e-01   5.214 1.85e-07 ***
## NeighborhoodGreens   8.154e-01  1.255e+00   0.650 0.515868
## NeighborhoodGrnHill  1.681e+01  3.786e+02   0.044 0.964576
## NeighborhoodIDOTRR   2.839e+00  7.465e-01   3.803 0.000143 ***
## NeighborhoodLandmrk  1.681e+01  5.354e+02   0.031 0.974953
## NeighborhoodMeadowV  2.968e+00  8.273e-01   3.588 0.000334 ***
## NeighborhoodMitchel  3.233e+00  6.505e-01   4.970 6.69e-07 ***
## NeighborhoodNAmes    3.131e+00  6.176e-01   5.069 3.99e-07 ***
## NeighborhoodNoRidge  2.937e+00  6.706e-01   4.379 1.19e-05 ***
## NeighborhoodNPkVill  2.274e+00  9.797e-01   2.321 0.020302 *
## NeighborhoodNWAmes   4.057e+00  6.370e-01   6.369 1.91e-10 ***
## NeighborhoodOldTown  2.334e+00  6.908e-01   3.378 0.000729 ***
## NeighborhoodSawyer   4.144e+00  6.328e-01   6.549 5.79e-11 ***
## NeighborhoodSawyerW  2.901e+00  6.545e-01   4.433 9.31e-06 ***
## NeighborhoodSomerst  2.396e+00  6.420e-01   3.733 0.000189 ***
## NeighborhoodStoneBr  1.205e+00  7.621e-01   1.582 0.113721
## NeighborhoodSWISU    3.030e+00  8.124e-01   3.729 0.000192 ***
## NeighborhoodTimber   2.711e+00  6.730e-01   4.028 5.63e-05 ***
## NeighborhoodVeenker  3.099e+00  7.638e-01   4.057 4.98e-05 ***
## BldgType2fmCon      -9.491e-01  6.600e-01  -1.438 0.150435
## BldgTypeDuplex       1.512e-01  3.018e-01   0.501 0.616290
## BldgTypeTwnhs        8.349e-01  5.571e-01   1.499 0.133970
## BldgTypeTwnhsE       7.810e-01  2.767e-01   2.822 0.004774 **
```

Missing data imputation will proceed in the following way: * for columns with **irregular** lot shape, the lot frontage will be imputed with 0 * for columns with **regular** lot shape, imputation using a random forest will be performed

```
regs <- which(df0$LotShape != "Reg" & is.na(df0$LotFrontage))
df[regs, "LotFrontage"] <- df0[regs, "LotFrontage"] <- 0

for(col in colnames(df0)[c(1,2,4:80)])
  df0[[col]] <- na.fill(df0[[col]])
```

Before proceeding with model-based missing data imputation, we ought to divide the dataset in train and test, as to not contaminate the test data and preserve its independence with the training data.

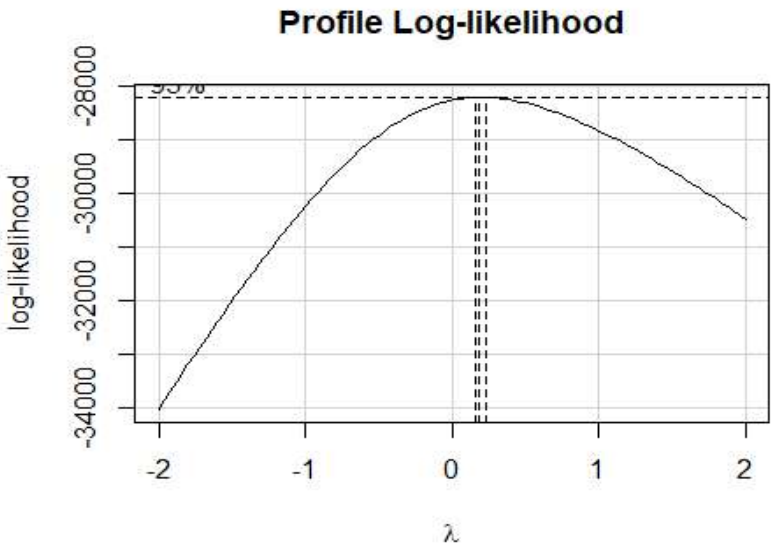
```
rm(list = ls(all = F) |> setdiff(c("df", "df0")))
set.seed(42)

train.rows <- sample(1:nrow(df), replace = F, size = .7 * nrow(df))
df.train <- df[train.rows, ]
df.test <- df[-train.rows, ]

dummyVars(~ ., data = df0) |> predict(newdata = df0) |> data.frame() |>
select(-SalePrice) -> data
data.train <- data[train.rows, ]
data.test <- data[-train.rows, ]

data.train.fit <- data.train[!is.na(data.train$LotFrontage), ]
data.train.imp <- data.train[is.na(data.train$LotFrontage), ]
data.test.imp <- data.test[is.na(data.test$LotFrontage), ]

ctrl <- trainControl(method = "cv", number = 5, search = "grid")
rf <- train(LotFrontage ~ ., data = data.train.fit, method = "rf",
            tuneGrid = expand.grid(.mtry = 5:7), ntree = 100, trControl =
ctrl)

train.imp <- predict(rf, newdata = data.train.imp)
test.imp <- predict(rf, newdata = data.test.imp)

df.train[is.na(df.train$LotFrontage), "LotFrontage"] <- train.imp
df.test[is.na(df.test$LotFrontage), "LotFrontage"] <- test.imp

rm(list = ls(all = F) |> setdiff(c("df.train", "df.test")))
```

Target variable

```
p1 <- ggplot(df.train, aes(x=SalePrice)) +
  geom_histogram(fill="red", bins=100) + ggtitle("SalePrice Distribution")
p1
```

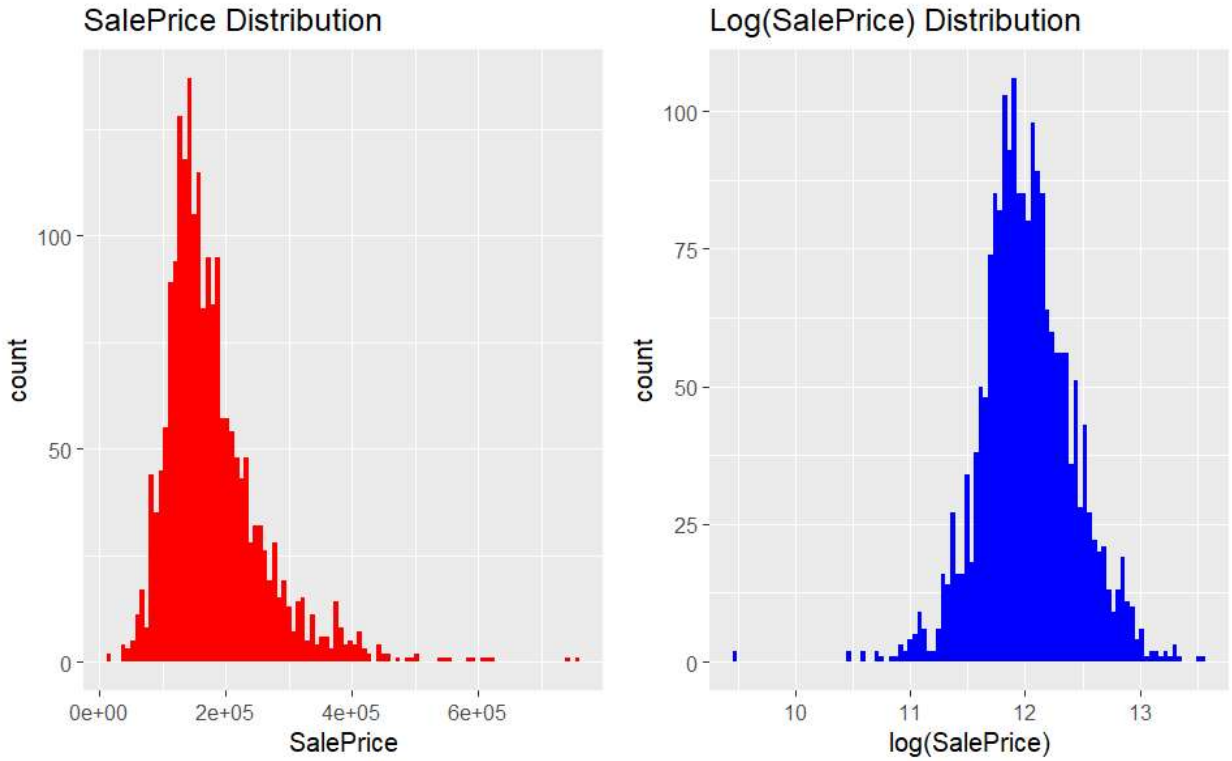
We can observe that the target variable is right skewed: we will check the most appropriate Box-Cox transformation to make it more normally distributed.

```
car::boxCox(lm(SalePrice ~ ., data = df.train))
```



The optimal value for lambda appears to be 0: we will therefore apply a logarithmic transformation. What follows is a before-after comparison:

```
p2 <- ggplot(df.train, aes(x=log(SalePrice))) +
  geom_histogram(fill="blue", bins=100) + ggtitle("Log(SalePrice)
Distribution")

grid.arrange(p1, p2, ncol=2)
```



```
df.train$SalePrice <- log(df.train$SalePrice)
df.test$SalePrice <- log(df.test$SalePrice)
```

Non-zero-variance variables

```
near_zero_var <- nearZeroVar(df.train, freqCut = 19, uniqueCut = 10,
allowParallel = TRUE, saveMetrics = TRUE)
near_zero_var
```

```
##                 freqRatio percentUnique zeroVar   nzv
## LowQualFinSF   667.666667     1.2795276   FALSE  TRUE
## KitchenAbvGr    23.743902     0.1968504   FALSE  TRUE
## EnclosedPorch  106.937500     7.0866142   FALSE  TRUE
## X3SsnPorch     669.666667     0.9350394   FALSE  TRUE
## ScreenPorch    185.200000     4.8720472   FALSE  TRUE
## PoolArea      2023.000000     0.4921260   FALSE  TRUE
## MiscVal        130.866667     1.4763780   FALSE  TRUE
```

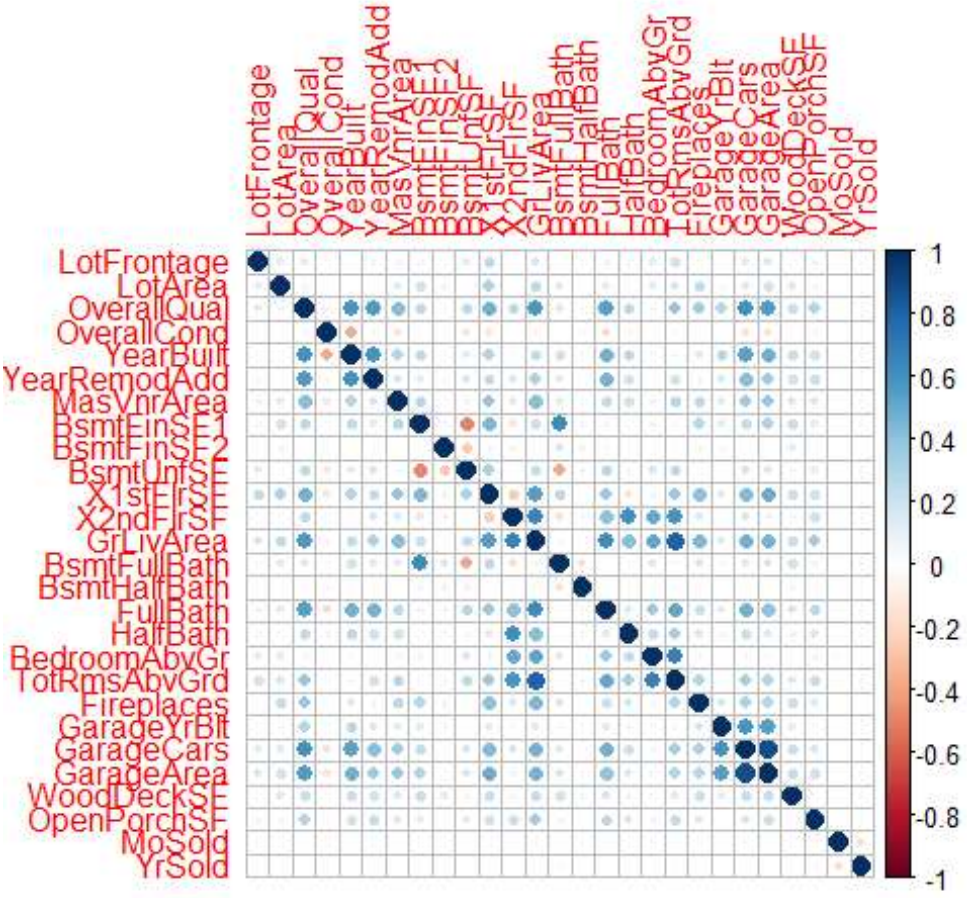
```
df.train <- df.train[, !near_zero_var$nzv]
df.test <- df.test[, !near_zero_var$nzv]
dim(df.train)
```

```
## [1] 2032   28
```

Correlation Analysis

In the following section, we will check the raw correlation and partial between the predictors.

```
X_raw <- df.train[, names(df.train) != "SalePrice"]
y_raw <- df.train$SalePrice

corrplot::corrplot(cor(X_raw))
```



We can see clusters of variables that are more correlated, particularly variables **garage** (“GarageArea”, “GarageCars”, “GarageYrBlt”). Also, one can notice how variables “OverallQlt” and “GrLivArea” appear to be slightly correlated with almost all other predictors.

Matrix rank

Before continuing, we want to be sure that our matrix X is full rank, and remove linear dependent columns otherwise.

```
rankMatrix(X_raw)
```

```
## [1] 27
## attr(,"method")
## [1] "tolNorm2"
## attr(,"useGrad")
## [1] FALSE
## attr(,"tol")
## [1] 4.511946e-13
```

```
dim(X_raw)
```

```
## [1] 2032   27
```

```
X_clean=X_raw
```

Our matrix is full rank.

High correlation check

Now, we check for highly correlated columns, removing all variables that are shown to be more correlated to another predictor with a correlation stronger or equal than 0.8 (absolute value). We also obtain the Variance Inflation Factor for each variable, to better understand how correlation amongst predictors adds to the variance.

```
vif_values <-vif(lm(y_raw~.,data=X_clean))
print(vif_values[vif_values>10])
```

```
## X1stFlrSF X2ndFlrSF GrLivArea
##  75.39916  91.77969 123.74567
```

```
cor_matrix_clean <- cor(X_clean)
high.corrs <- findCorrelation(cor_matrix_clean, cutoff=0.8)
high.corrs <- colnames(X_clean[high.corrs])
print(high.corrs)
```

```
## [1] "GrLivArea"  "GarageCars"
```

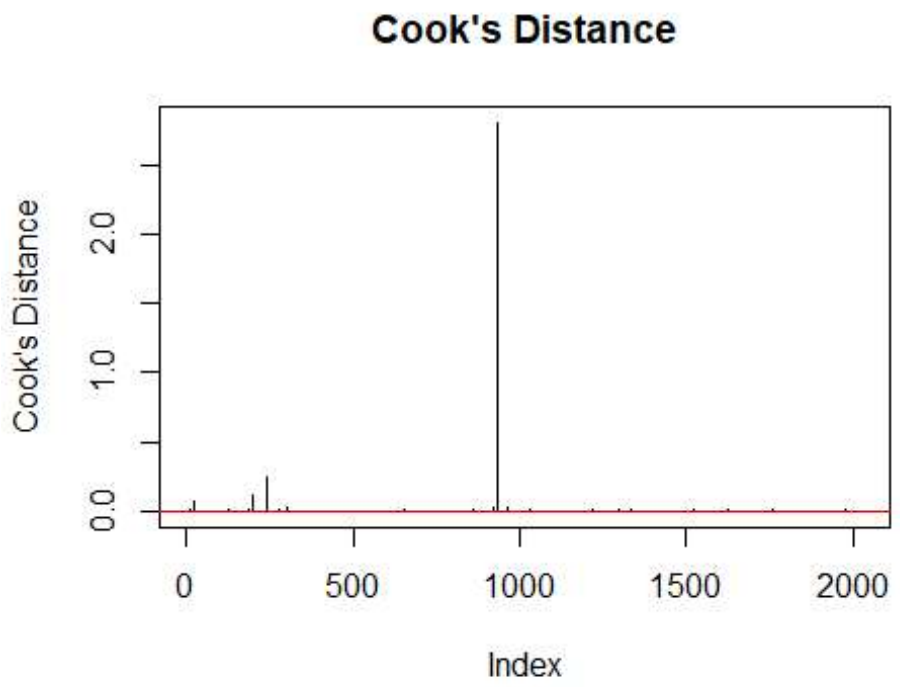
```
if(length(high.corrs) > 0){
  X_clean <- X_clean |> select(-all_of(high.corrs))
  df.train <- df.train |> select(-all_of(high.corrs))
  df.test <- df.test |> select(-all_of(high.corrs))
}
```

Our matrix has 2032 rows and 25 columns.

Outlier detection

We will search for outliers using Cook Distance, and remove rows that appear potentially problematic.

```
lm_cook <- lm(y_raw ~ ., data = X_clean)
cookdis <- cooks.distance(lm_cook)

plot(cookdis, type="h", main="Cook's Distance", ylab="Cook's Distance")
abline(h = 4/(nrow(X_clean)-ncol(X_clean)), col="red")
```




```
outlier_indices <- which(cookdis > 4/(nrow(X_clean) - ncol(X_clean)))

X_clean <- X_clean[-outlier_indices, ]
dim(X_clean)

## [1] 1937   25

y_clean <- y_raw[-outlier_indices]
```

Now, matrix X has 1937 rows and 25 columns.

Centering and Scaling

To use the Knockoff technique, we need our data to be centered and scaled.

```
normalize <- function(data) as.matrix(data) |> scale() |> apply(2,
function(t) t / sqrt(length(t) - 1))

X <- normalize(X_clean)
y <- normalize(y_clean)
```

Hypothesis check

Finally, we must ensure the following hypothesis to be satisfied.

- The sum of squares should be 1:

```
unique(apply(X, 2, function(col) sum(col^2)))
```

```
## [1] 1 1 1 1
```

- Means should be 0:

```
unique(apply(X, 2, function(col) round(sum(col), 10)))
```

```
## [1] 0
```

- The Fixed-X Method requires that:

```
nrow(X) >= 2 * ncol(X)
```

```
## [1] TRUE
```

- Finally, matrix X must be full rank:

```
unlist(rankMatrix(X))[1]
```

```
## [1] 25
```

This guarantees that we can use the Fixed-X approach of the Knockoff Filter.

Fixed-X Knockoff

Now, we're going to perform the Fixed-X Knockoff technique, and estimate the False Discovery Rate after selecting the most significant predictors through a Lasso selection.

```
rm(list = ls(all = F) |> setdiff(c("df.train", "df.test", "X", "y")))

n <- nrow(X)
p <- ncol(X)
varType <- rep("N",p)
```

Knockoff construction

First, we obtain matrix Σ and its inverse, Σ^{-1}.

```
Sigma <- crossprod(X)
Sigma_inv <- solve(crossprod(X))
```

We construct U so that $U^t U = I_p$ and $U^t X = 0_p$. In order to do so, we conjoin X by row with a zero-matrix of the same dimensions, obtaining X_0 of size $n \times 2\,p$.

By computing the SVD of X_0 and taking the last p columns of the obtained U matrix, we get a matrix that is orthonormal by construction, and whose column space is orthogonal to the column space of X (which coincides with the column space of the first p columns of U).

```
X_svd <- X |> cbind(matrix(0,n,p)) |> svd()
U <- X_svd$u[,(p+1):(2*p)]
```

Now that we have Σ, Σ^{-1} and U, we can create knockoff variables. First, we'll use the Equi-Correlated Knockoffs method. To obtain the maximum possible value of d, we use the formula:

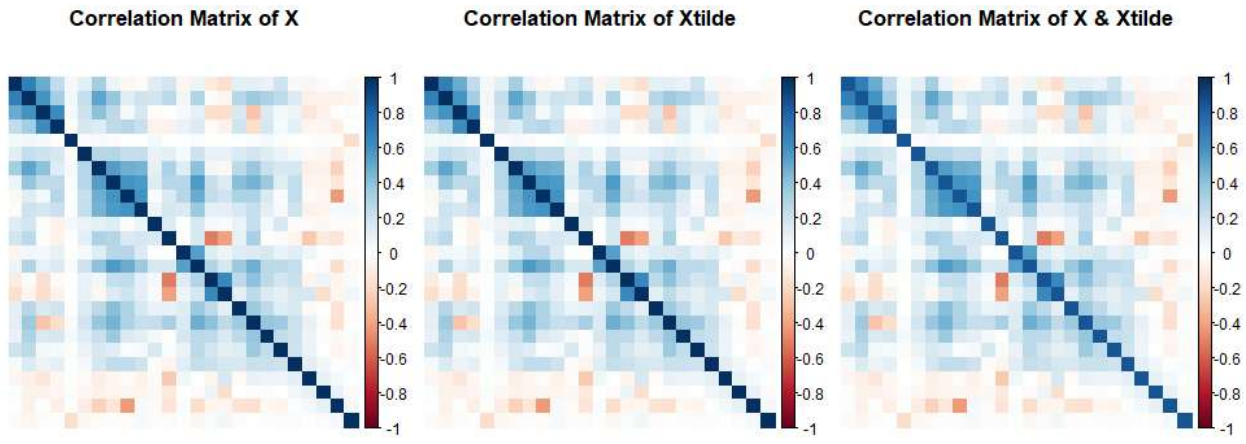
```
d <- min(c(2 * svd(X)$d^2, 1)) * (1 - 1e-7)
d

## [1] 0.1495731

d <- diag(rep(d, p))

C <- chol(d %*% (2 * diag(rep(1, p)) - Sigma_inv %*% d))
Xtilde <- X %*% (diag(rep(1, p)) - Sigma_inv %*% d) + U %*% C
```

We obtain d = 0.1496.

Plots about knockoff variables

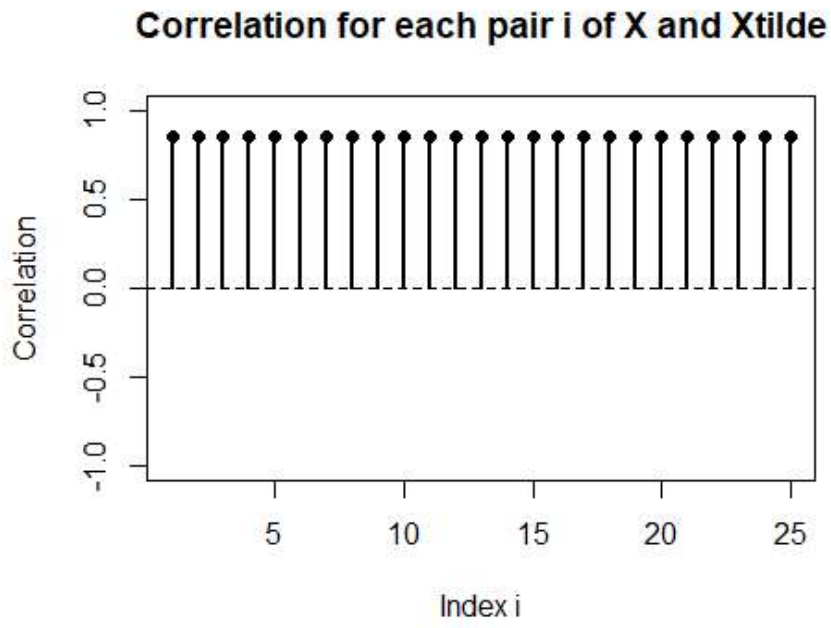
These plots show the correlation matrix of X and the correlation matrix of $\widetilde{X}$, which should be identical. The third plot shows the correlation matrix between X and $\widetilde{X}$ (note that the diagonal represents 1-d).

```
par(mfrow = c(1,3))
plot1 <- corrplot(crossprod(X),
          method = "color",
          order = "hclust",
          tl.pos = "n",
          addgrid.col = NA,
          title = "Correlation Matrix of X",
          mar = c(0,0,1,0))

plot2 <- corrplot(crossprod(Xtilde),
          method = "color",
          order = "hclust",
          tl.pos = "n",
          addgrid.col = NA,
          title = "Correlation Matrix of Xtilde",
          mar = c(0,0,1,0))

plot3 <- corrplot(crossprod(X,Xtilde),
          method = "color",
          order = "hclust",
          tl.pos = "n",
          addgrid.col = NA,
          title = "Correlation Matrix of X & Xtilde",
          mar = c(0,0,1,0))
```



This plot will show us correlation between each pair of variables X_i and $\widetilde{X}_i$.

```r
cor_cols <- sapply(seq_len(ncol(X)), function(i) {
 x <- X[, i]
 xtilde <- Xtilde[, i]
 cor(x, xtilde, use = "pairwise.complete.obs")
})

par(mfrow = c(1,1))
plot(cor_cols, type = "h", lwd = 2,
 xlab = "Index i", ylab = "Correlation",
 main = "Correlation for each pair i of X and Xtilde",
 ylim = c(-1, 1))
points(cor_cols, pch = 16)
abline(h = 0, lty = 2)
```



Knockoff Statistic

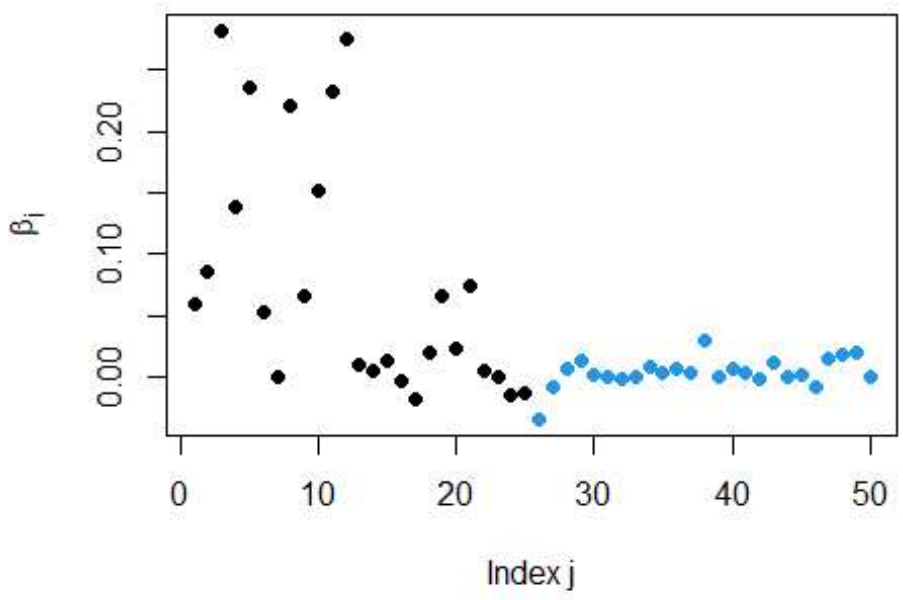
We combine X and $\widetilde{X}$, fit a Lasso using glmnet and find the best lambda through cross-validation.

```r
XX <- cbind(X,Xtilde)
fit <- glmnet(XX, y)
l <- (cv.glmnet(XX, y)$lambda.min)
```

This plot shows the estimates of the coefficients; blue variables are knockoff variables.

```r
cols <- c(rep(1, p), rep(4, p))
plot(1:(2*p), coef(fit, s=l)[-1],
     xlab="Index j",
```

```
     ylab=expression(hat(beta)[j]),
     pch=19,
     col=cols)
```



We want to find out which of the selected variables are knockoffs (labeled as K) and which are actual columns of $\widetilde{X}$ (labeled as N). The Lasso should pick as few knockoff variables as possible. We won't consider the intercept.

```
varType <- c(rep("N",p),rep("K",p))
S_hat <- which(coef(fit, s=l)[-1] != 0)
print(S_hat)
```

```
##  [1]  1  2  3  4  5  6  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 24 25
26 27
## [26] 28 29 30 32 34 35 36 37 38 40 41 42 43 45 46 47 48 49
```

```
table(varType[S_hat])
```

```
##  K  N
## 20 23
```

Now, we calculate the Knockoff Statistic for each pair of variables i. First, we find the first lambda for which a variable enters the model (if it's null, we put 0), then calculate W statistic for each pair of variables.

We can calculate Z:

```
first_nonzero <- function(x) match(T, abs(x) > 0)
indices <- apply(fit$beta, 1, first_nonzero)
Z = ifelse(is.na(indices), 0, fit$lambda[indices]* n)
```

```
LotFrontage   LotArea    OverallQual  OverallCond  YearBuilt    YearRemodAdd
1.29135294   6.27933483  33.51092134   2.98319277  10.97331594   9.11023968

MasVnrArea   BsmtFinSF1 BsmtFinSF2    BsmtUnfSF    X1stFlrSF     X2ndFlrSF
0.00000000   8.30091109 0.97686114    0.81100728   19.17618125   4.32810192

BsmtFullBath BsmtHalfBath  FullBath     HalfBath   BedroomAbvGr TotRmsAbvGrd
1.41725825   0.22047949    13.21739790  0.041313   0.18304594   12.04320070

Fireplaces  GarageYrBlt   GarageArea   WoodDeckSF  OpenPorchSF    MoSold
8.30091109   1.55543918  17.47262216   2.71817415   0.00000000    0.11495819

YrSold     LotFrontage.    LotArea.   OverallQual.  OverallCond.  YearBuilt.
0.5589958  0.24197596     0.07923622  23.09777821   0.46408818    0.07923622

YearRemodAdd.  MasVnrArea.  BsmtFinSF1.  BsmtFinSF2.  BsmtUnfSF.   X1stFlrSF.
4.75008652   0.05461445     0.00000000   0.18304594   0.05461445   2.05619906

X2ndFlrSF.  BsmtFullBath.  BsmtHalfBath.  FullBath.   HalfBath. BedroomAbvGr.
0.04534188  2.47669905     0.00000000    12.04320070  0.29146090  0.04534188

TotRmsAbvGrd. Fireplaces.  GarageYrBlt. GarageArea. WoodDeckSF.  OpenPorchSF.
12.04320070   0.00000000   0.22047949   7.56348103  3.94360528   2.05619906

MoSold.      YrSold.
0.46408818   0.00000000
```

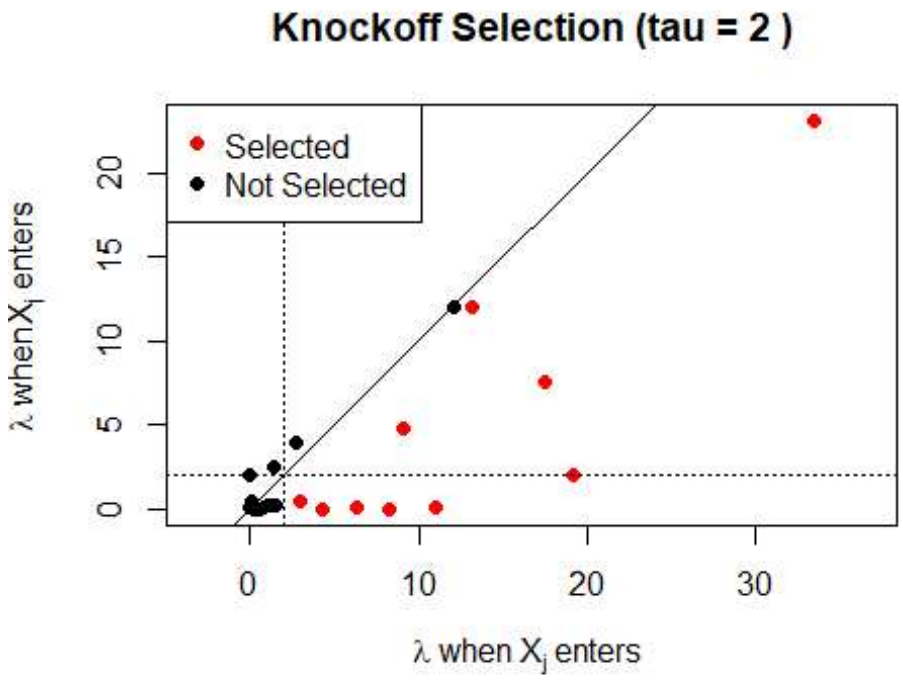
And then W:

```
orig = 1:p
W = pmax(Z[orig], Z[orig+p]) * sign(Z[orig] - Z[orig+p])
W

LotFrontage  LotArea  OverallQual  OverallCond  YearBuilt  YearRemodAdd
1.29135      6.27933  33.51092     2.98319      10.97331   9.11023

MasVnrArea   BsmtFinSF1   BsmtFinSF2   BsmtUnfSF  X1stFlrSF   X2ndFlrSF
-0.05461     8.30091      0.97686      0.81100    19.17618    4.32810192

BsmtFullBath  BsmtHalfBath  FullBath   HalfBath   BedroomAbvGr TotRmsAbvGrd
-2.47669      0.22047949    13.21739   -0.29146   0.18304      0.00000000

Fireplaces  GarageYrBlt   GarageArea   WoodDeckSF   OpenPorchSF    MoSold
8.30091     1.55543       17.47262     -3.94360     -2.05619       -0.46408
YrSold
0.55899
```

FDP Estimate

First, let's see an example where we choose tau=2:

```
tau = 2

my_cols <- ifelse(W >= tau, "red", "black")
plot(Z[orig], Z[orig+p], col=my_cols, pch=19, asp=1,
     xlab=expression(paste(lambda," when ",X[j]," enters ")),
     ylab=expression(paste(lambda," when ",tilde(X)[j]," enters ")),
     main=paste("Knockoff Selection (tau =", tau, ")"))
abline(a=0,b=1)
abline(h=tau,lty=3)
abline(v=tau,lty=3)
legend("topleft", legend=c("Selected", "Not Selected"), col=c("red",
"black"), pch=19)
```



The model selects all variables on the right of the vertical line (tau=2) and below the diagonal (positive W).

```
S_hat_tau = which(W >= tau)
S_hat_tau

LotArea  OverallQual  OverallCond    YearBuilt YearRemodAdd   BsmtFinSF1
      2            3            4            5            6            8
X1stFlrSF    X2ndFlrSF     FullBath   Fireplaces   GarageArea
       11           12           15           19           21
```

```r
table(varType[S_hat_tau])
```

```
##  N
## 11
```

```r
(1 + sum(W <= -tau)) / length(S_hat_tau)
```

```
## [1] 0.3636364
```

These are the 12 selected variables. We obtain an FDP estimate of 0.36 by putting tau=2.

To control our estimate of the FDP, we can chose alpha as the maximum tolerable error; then, the algorithm automatically chooses the best value of tau such that the FDP estimate is lower than alpha.

For each possible tau, we find the expected FDP.

```r
taus = sort(c(0,abs(W)))

FDP_hat = sapply(taus, function(tau)
  (1 + sum(W <= -tau)) / sum(W >= tau))
FDP_hat
```

```
TotRmsAbvGrd  MasVnrArea  BedroomAbvGr  BsmtHalfBath    HalfBath      MoSold
0.4210526     0.4210526   0.3888889     0.3333333       0.3529412     0.3750000

YrSold      BsmtUnfSF    BsmtFinSF2   LotFrontage  GarageYrBlt   OpenPorchSF
0.3125000   0.2500000    0.2666667    0.2857143    0.3076923     0.3333333

BsmtFullBath  OverallCond   WoodDeckSF    X2ndFlrSF    LotArea     BsmtFinSF1
0.3636364     0.2727273     0.1818182     0.2000000    0.1000000   0.1111111

Fireplaces   YearRemodAdd   YearBuilt    FullBath     GarageArea   X1stFlrSF
0.1250000    0.1250000      0.1666667    0.2000000    0.2500000    0.5000000

OverallQual
1.0000000
```

Now, we can choose a value of alpha and find the best tau for that value.

```r
alpha = 0.1
tau_hat <- taus[[which.min(FDP_hat[FDP_hat  <= alpha]) |> names()]]

print(paste("Selected tau:", tau_hat))
```

```
## [1] "Selected tau: 4.32810192369755"
```

```r
S_hat = which(W >= tau_hat)
print(paste("Estimated FDP:", round((1 + sum(W <= -tau_hat)) / length(S_hat),
3)))
```

```
## [1] "Estimated FDP: 0.091"
```

The selected tau is 4.3281, which generates an estimated FDP of 0.091.

```
equi.vars <- colnames(X)[which(W >= tau_hat)]
table(varType[S_hat])
```

```
##  N
## 10
```

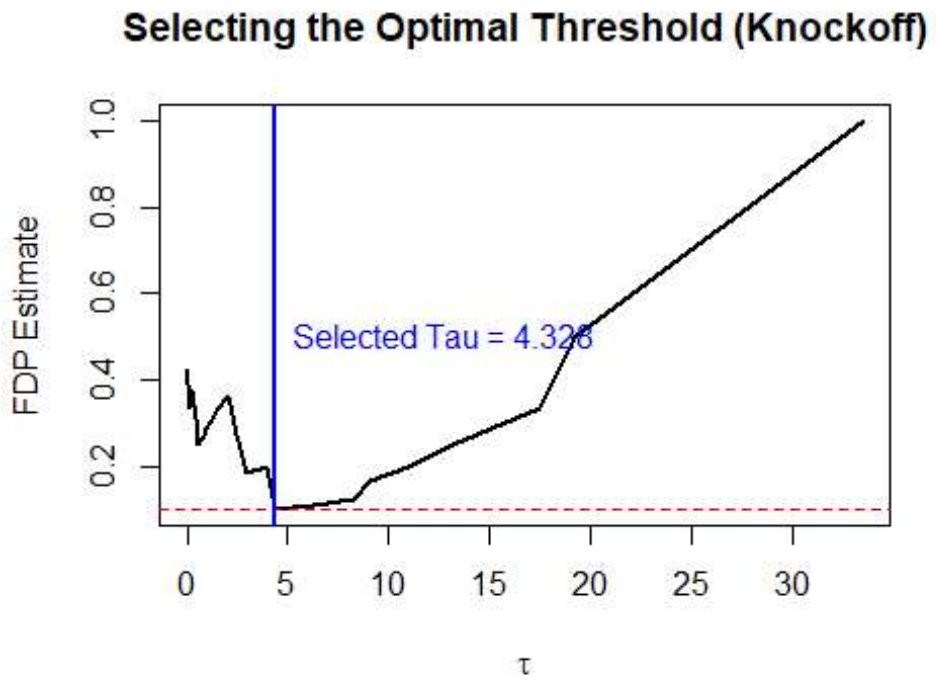
```
print(equi.vars)
```

```
##  "LotArea"      "OverallQual"  "YearBuilt"    "YearRemodAdd" "BsmtFinSF1"
"X1stFlrSF"    "X2ndFlrSF"    "FullBath"     "Fireplaces"   "GarageArea"
```

We selected the following 10 variables.

This plot shows our results.

```
plot(taus, FDP_hat, type="l", lwd=2,
     xlab=expression(tau), ylab="FDP Estimate",
     main="Selecting the Optimal Threshold (Knockoff)")
abline(h = alpha, col="red", lty=2)
abline(v = tau_hat, col="blue", lwd=2)
text(x=tau_hat, y=0.5, labels=paste("Selected Tau =", round(tau_hat, 3)),
pos=4, col="blue")
```



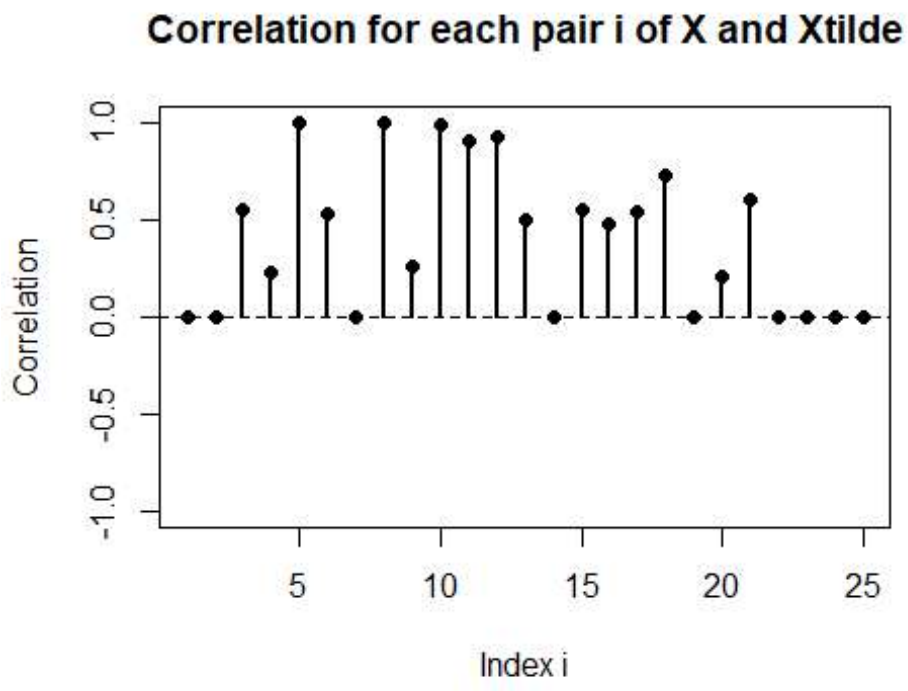
Knockoff SDP

We could also use Semidefinite-Programming Knockoffs by using function *create.fixed* from package **knockoffs**.

```
Xtilde_sdp <- create.fixed(X,method="sdp")$Xk
```

We observe the correlation between each pair of variables from X and X_tilde, now using SDP-Knockoffs.

```
cor_cols <- sapply(seq_len(ncol(X)), function(i) {
  x <- X[, i]
  xtilde <- Xtilde_sdp[, i]
  cor(x, xtilde, use = "pairwise.complete.obs")
})

par(mfrow = c(1,1))
plot(cor_cols, type = "h", lwd = 2,
     xlab = "Index i", ylab = "Correlation",
     main = "Correlation for each pair i of X and Xtilde",
     ylim = c(-1, 1))
points(cor_cols, pch = 16)
abline(h = 0, lty = 2)
```



Now, correlation between X and $\widetilde{X}$ is different for each variable.

We repeat the process of fitting a Lasso and estimating the Knockoff Statistic W, using the SDP-Knockoffs:

```
XX_sdp <- cbind(X, Xtilde_sdp)

fit_sdp <- glmnet(XX_sdp, y)

indices_sdp <- apply(fit_sdp$beta, 1, first_nonzero)

Z_sdp = ifelse(is.na(indices_sdp), 0, fit_sdp$lambda[indices_sdp] * n)

W_sdp = pmax(Z_sdp[orig], Z_sdp[orig+p]) * sign(Z_sdp[orig] - Z_sdp[orig+p])
```

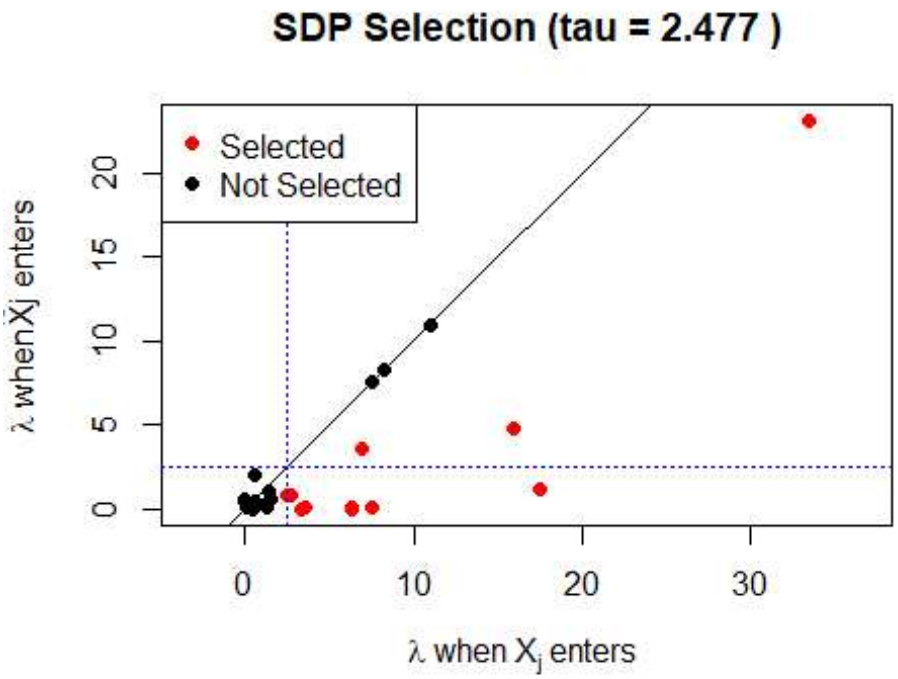
And, for alpha=0.1, find the value of tau and the best estimate of FDP.

```
taus_sdp = sort(c(0, abs(W_sdp)))

FDP_hat_sdp = sapply(taus_sdp, function(tau)
  (1 + sum(W_sdp <= -tau)) / max(1, sum(W_sdp >= tau)))

alpha = 0.1
tau_hat_sdp <- taus_sdp[[which.min(FDP_hat_sdp[FDP_hat_sdp <= alpha]) |>
names()]]
S_hat_sdp = which(W_sdp >= tau_hat_sdp)

print(paste("SDP tau:", tau_hat_sdp))

## [1] "SDP tau: 2.47669904745383"

print(paste("Estimated FDP:", round((1 + sum(W_sdp <= -tau_hat_sdp)) /
length(S_hat_sdp), 3)))

## [1] "Estimated FDP: 0.091"

selected_indices_sdp <- which(W_sdp >= tau_hat_sdp)
selected_names_sdp <- colnames(X)[selected_indices_sdp]

print(paste("Number of variables selected by SDP:",
length(selected_names_sdp)))

## [1] "Number of variables selected by SDP: 11"

print(selected_names_sdp)

##  "LotArea"  "OverallQual"  "OverallCond"  "YearRemodAdd"   "X1stFlrSF"
"X2ndFlrSF" "BsmtFullBath" "FullBath" "Fireplaces" "GarageArea" “WoodDeckSF"
```

We obtain tau=2.4767, which returns an estimated FDP of 0.091 and selects 11 variables. This plot better shows the results we obtained.

```
cols_sdp <- ifelse(W_sdp >= tau_hat_sdp, "red", "black")
```

```r
plot(Z_sdp[orig], Z_sdp[orig+p], col=cols_sdp, pch=19, asp=1,
     xlab=expression(paste(lambda," when ",X[j]," enters ")),
     ylab=expression(paste(lambda," when ",tilde(X),"j enters ")),
     main=paste("SDP Selection (tau =", round(tau_hat_sdp, 3), ")"))
abline(a=0,b=1)
abline(h=tau_hat_sdp, lty=3, col="blue")
abline(v=tau_hat_sdp, lty=3, col="blue")
legend("topleft", legend=c("Selected", "Not Selected"), col=c("red",
"black"), pch=19)
```



Stability Selection

We will do 100 iterations of the knockoff procedure. We decide to insert a variable into the final model if it gets selected more than 80% of the times.

```r
set.seed(42)

M <- 100
alpha <- 0.10
cutoff <- 0.8

selection_matrix <- matrix(0, nrow = ncol(X), ncol = M)
rownames(selection_matrix) <- colnames(X)
```

The cycle iterates the following steps:

- Creates $\widetilde{X}$ (using SDP-Knockoff to maximise performance). By setting “Randomize” to TRUE, it finds a plane generated by a random orthonormal base of vectors which are orthogonal to the column space of X.
- Calculates the knockoff statistics.
- Finds tau such that FDP is lower than alpha.
- Stores the selected variables.

```
for(i in 1:M) {

  Xtilde_random <- create.fixed(X, method = "sdp", randomize = TRUE)$Xk

  XX_loop <- cbind(X, Xtilde_random)
  fit_loop <- glmnet(XX_loop, y)
  indices_loop <- apply(fit_loop$beta, 1, first_nonzero)
  Z_loop = ifelse(is.na(indices_loop), 0, fit_loop$lambda[indices_loop] * n)
  W_loop = pmax(Z_loop[orig], Z_loop[orig+p]) * sign(Z_loop[orig] -
Z_loop[orig+p])

  taus_loop = sort(c(0, abs(W_loop)))
  FDP_hat_loop = sapply(taus_loop, function(tau)
    (1 + sum(W_loop <= -tau)) / max(1, sum(W_loop >= tau)))
  tau_hat_loop <- taus_loop[which(FDP_hat_loop <= alpha)[1]]

  if(!is.na(tau_hat_loop)) {
    selected_indices <- which(W_loop >= tau_hat_loop)

    if(length(selected_indices) > 0){
      selection_matrix[names(selected_indices), i] <- 1
    }
  }

  cat(paste("Iteration", i, ": Selected", sum(selection_matrix[,i]),
"variables.\n"))
}
```

```
## Iteration 1 : Selected 12 variables.
## Iteration 2 : Selected 15 variables.
## Iteration 3 : Selected 11 variables.
## Iteration 4 : Selected 16 variables.
## Iteration 5 : Selected 12 variables.
## Iteration 6 : Selected 15 variables.
## Iteration 7 : Selected 11 variables.
## Iteration 8 : Selected 11 variables.
## Iteration 9 : Selected 0 variables.
## Iteration 10 : Selected 11 variables.
## Iteration 11 : Selected 12 variables.
## Iteration 12 : Selected 12 variables.
```

```
## Iteration 13 : Selected 11 variables.
## Iteration 14 : Selected 16 variables.
## Iteration 15 : Selected 15 variables.
## Iteration 16 : Selected 15 variables.
## Iteration 17 : Selected 12 variables.
## Iteration 18 : Selected 12 variables.
## Iteration 19 : Selected 13 variables.
## Iteration 20 : Selected 12 variables.
## Iteration 21 : Selected 15 variables.
## Iteration 22 : Selected 14 variables.
## Iteration 23 : Selected 20 variables.
## Iteration 24 : Selected 20 variables.
## Iteration 25 : Selected 12 variables.
## Iteration 26 : Selected 11 variables.
## Iteration 27 : Selected 0 variables.
## Iteration 28 : Selected 12 variables.
## Iteration 29 : Selected 12 variables.
## Iteration 30 : Selected 12 variables.
## Iteration 31 : Selected 13 variables.
## Iteration 32 : Selected 0 variables.
## Iteration 33 : Selected 13 variables.
## Iteration 34 : Selected 13 variables.
## Iteration 35 : Selected 0 variables.
## Iteration 36 : Selected 10 variables.
## Iteration 37 : Selected 0 variables.
## Iteration 38 : Selected 15 variables.
## Iteration 39 : Selected 15 variables.
## Iteration 40 : Selected 12 variables.
## Iteration 41 : Selected 13 variables.
## Iteration 42 : Selected 13 variables.
## Iteration 43 : Selected 15 variables.
## Iteration 44 : Selected 0 variables.
## Iteration 45 : Selected 16 variables.
## Iteration 46 : Selected 0 variables.
## Iteration 47 : Selected 12 variables.
## Iteration 48 : Selected 12 variables.
## Iteration 49 : Selected 11 variables.
## Iteration 50 : Selected 12 variables.
## Iteration 51 : Selected 16 variables.
## Iteration 52 : Selected 13 variables.
## Iteration 53 : Selected 13 variables.
## Iteration 54 : Selected 14 variables.
## Iteration 55 : Selected 16 variables.
## Iteration 56 : Selected 11 variables.
## Iteration 57 : Selected 12 variables.
## Iteration 58 : Selected 14 variables.
## Iteration 59 : Selected 12 variables.
## Iteration 60 : Selected 13 variables.
## Iteration 61 : Selected 16 variables.
## Iteration 62 : Selected 13 variables.
```

```
## Iteration 63 : Selected 13 variables.
## Iteration 64 : Selected 17 variables.
## Iteration 65 : Selected 13 variables.
## Iteration 66 : Selected 11 variables.
## Iteration 67 : Selected 14 variables.
## Iteration 68 : Selected 14 variables.
## Iteration 69 : Selected 12 variables.
## Iteration 70 : Selected 15 variables.
## Iteration 71 : Selected 17 variables.
## Iteration 72 : Selected 14 variables.
## Iteration 73 : Selected 15 variables.
## Iteration 74 : Selected 0 variables.
## Iteration 75 : Selected 12 variables.
## Iteration 76 : Selected 20 variables.
## Iteration 77 : Selected 13 variables.
## Iteration 78 : Selected 12 variables.
## Iteration 79 : Selected 20 variables.
## Iteration 80 : Selected 12 variables.
## Iteration 81 : Selected 12 variables.
## Iteration 82 : Selected 13 variables.
## Iteration 83 : Selected 15 variables.
## Iteration 84 : Selected 14 variables.
## Iteration 85 : Selected 12 variables.
## Iteration 86 : Selected 13 variables.
## Iteration 87 : Selected 15 variables.
## Iteration 88 : Selected 12 variables.
## Iteration 89 : Selected 13 variables.
## Iteration 90 : Selected 20 variables.
## Iteration 91 : Selected 13 variables.
## Iteration 92 : Selected 12 variables.
## Iteration 93 : Selected 0 variables.
## Iteration 94 : Selected 12 variables.
## Iteration 95 : Selected 13 variables.
## Iteration 96 : Selected 16 variables.
## Iteration 97 : Selected 20 variables.
## Iteration 98 : Selected 14 variables.
## Iteration 99 : Selected 11 variables.
## Iteration 100 : Selected 14 variables.
```

The stabilised variables are:

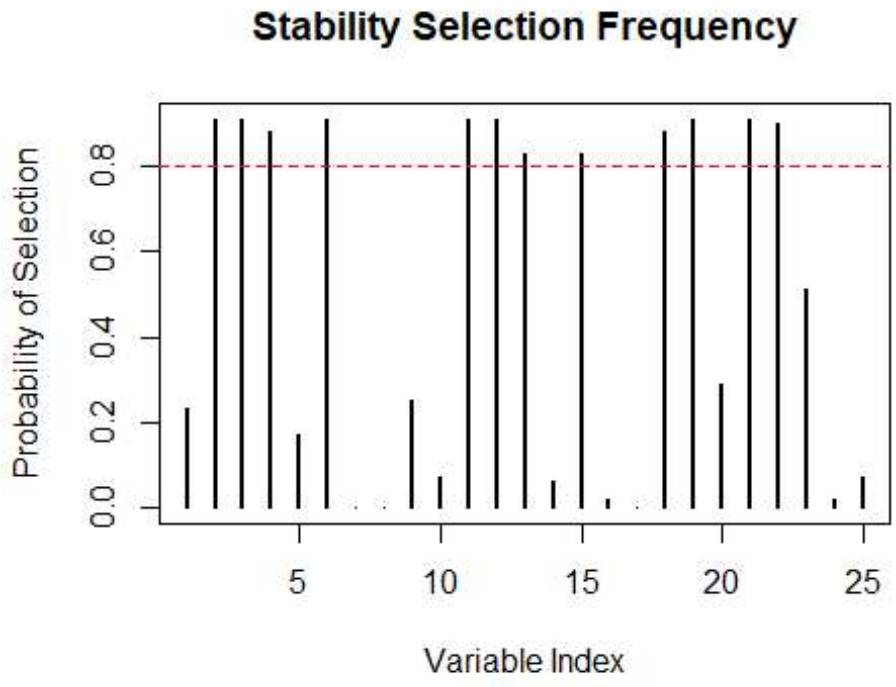
```
probs <- rowMeans(selection_matrix)

stable.vars <- names(probs)[probs >= cutoff]

print(paste("Count:", length(stable.vars)))

## [1] "Count: 12"
```

```
print(stable.vars)
```

```
## "LotArea"  "OverallQual"  "OverallCond"  "YearRemodAdd" "X1stFlrSF"
"X2ndFlrSF" "BsmtFullBath" "FullBath" "TotRmsAbvGrd" "Fireplaces"
“GarageArea" "WoodDeckSF"
```

This graph represents, for each variable, the probability of being selected.

```
plot(probs, type="h", lwd=2,
     main="Stability Selection Frequency",
     ylab="Probability of Selection", xlab="Variable Index")
abline(h=cutoff, col="red", lty=2)
```



Random Forest

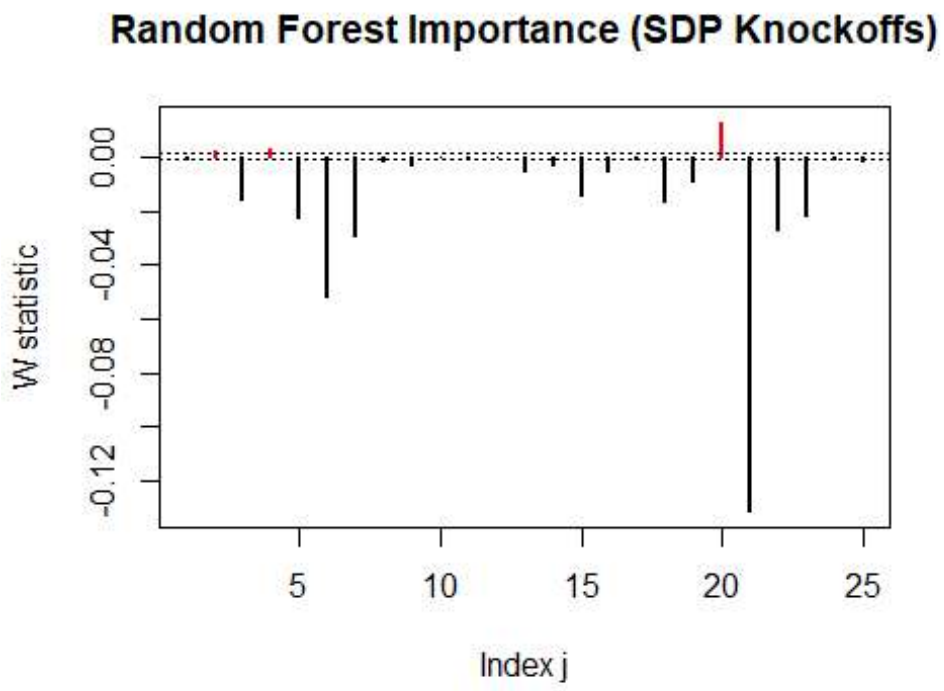
The Knockoff Filter is just a framework: it can be used with other variable selection techniques other than Lasso, such as Random Forests. We will show an example, in which we’ll use Variable Importance (“impurity”) to build the Knockoff Statistic, and use SDP-Knockoffs to gain a better performance.

```
set.seed(42)

XX_sdp_rf <- cbind(X, Xtilde_sdp)

random_forest_importance <- function(X, y, ...) {
  df = data.frame(y=y, X=X)
```

```
  rfFit = ranger::ranger(y~., data=df, importance="impurity", write.forest=F,
...)
  as.vector(rfFit$variable.importance)
}


Z_rf = random_forest_importance(XX_sdp_rf, y)


orig = 1:p
W_rf = abs(Z_rf[orig]) - abs(Z_rf[orig+p])
```

Initially, we select tau=0.001.

```
tau_rf = 0.001

my_cols_rf <- ifelse(W_rf >= tau_rf, "red", "black")

plot(W_rf, col=my_cols_rf, type="h", lwd=2, xlab="Index j",
     main="Random Forest Importance (SDP Knockoffs)", ylab="W statistic")
abline(h=tau_rf,lty=3)
abline(h=-tau_rf,lty=3)
```



For this selected tau, we obtain:

```
S_hat_tau_rf = which(W_rf >= tau_rf)
(1 + sum(W_rf <= -tau_rf)) / length(S_hat_tau_rf)
```

```
## [1] 6.666667
```

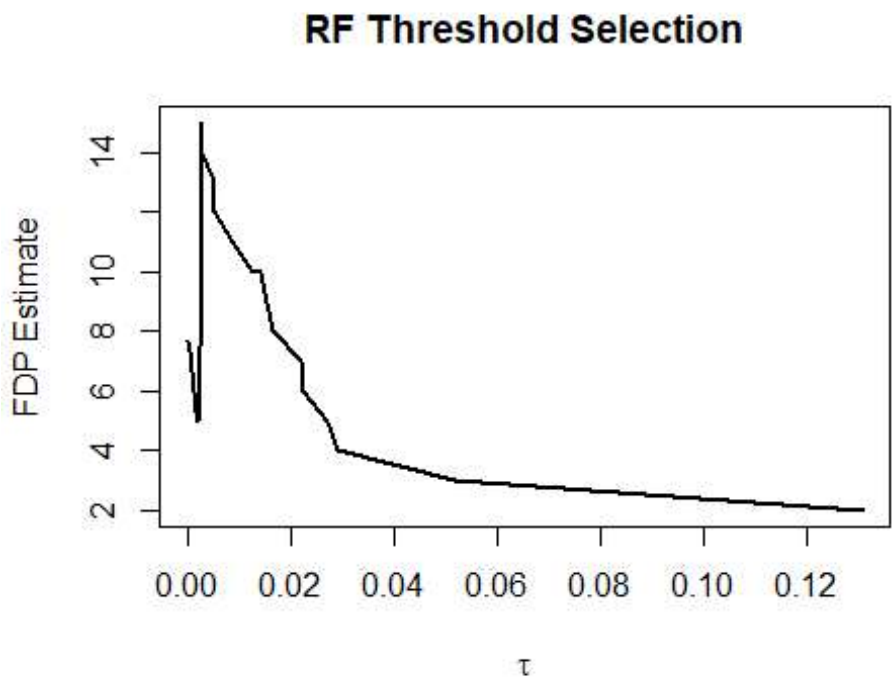
An estimated FDP of 6.666, which means that for each significant variable, the model inserts more than 7 false positive discoveries.

Now, we select alpha=0.1 and the algorithm finds the best tau.

```
taus = sort(c(0, abs(W_rf)))

FDP_hat_rf = sapply(taus, function(tau)
  (1 + sum(W_rf <= -tau)) / max(1, sum(W_rf >= tau)))

alpha = 0.1
tau_hat_rf <- taus[which(FDP_hat_rf <= alpha)[1]]

S_hat_rf = which(W_rf >= tau_hat_rf)
print(paste("Selected by RF:", length(S_hat_rf)))

## [1] "Selected by RF: 0"

print(colnames(X)[S_hat_rf])

## character(0)

(1 + sum(W_rf <= -tau_hat_rf)) / length(S_hat_rf)

## [1] NA
```

We see how the algorithm doesn't find any combination of variables that could grant an estimate of FDP lower than 0.1. This result can be express through this graph, that shows the FDP estimate for each possible tau:

```
plot(taus, FDP_hat_rf, type="l", lwd=2, xlab=expression(tau), ylab="FDP
Estimate",
     main="RF Threshold Selection")
abline(h=alpha, col="red", lty=2)
abline(v=tau_hat_rf, col="blue", lwd=2)
```



We can see how this model doesn't work with Fixed-X knockoff. However, this can be explained; Random Forest is not a linear model, because it places great importance on the true distribution of data. Since our predictors were fixed, its assumptions were violated and results lacked. This is one of the main reason Model-X Knockoffs are used.

Model Evaluation

We will compare the performances of 4 different models:

- a simple linear regression
- a LASSO regression
- a linear regression with equi-knockoff selected variables
- a linear regression with randomized SDP-knockoff selected variables

Firstly, the models are fit on the training data:

```
rm(list = setdiff(ls(all = F), c("df.train", "df.test", "equi.vars",
"stable.vars")))

X <- df.train |> select(-SalePrice) |> as.matrix()
y <- df.train$SalePrice

mod.lm <- lm(SalePrice ~ ., data = df.train)
mod.lasso <- glmnet(X, y); l <- cv.glmnet(X, y)$lambda.min
```

```
mod.ko.equi <- lm(SalePrice ~ ., data = df.train[, c(equi.vars,
"SalePrice")])
mod.ko.rand <- lm(SalePrice ~ ., data = df.train[, c(stable.vars,
"SalePrice")])
```

Then RMSE is evaluated on the test data:

```
X <- df.test |> select(-SalePrice) |> as.matrix()
y <- df.test$SalePrice

y.lm <- predict(mod.lm, newdata = df.test) |> exp()
y.lasso <- predict(mod.lasso, newx = X, s = 1) |> exp()
y.ko.equi <- predict(mod.ko.equi, newdata = df.test[, c(equi.vars,
"SalePrice")]) |> exp()
y.ko.rand <- predict(mod.ko.rand, newdata = df.test[, c(stable.vars,
"SalePrice")]) |> exp()
y <- exp(y)

c(linear = RMSE(y.lm, y),
  lasso = RMSE(y.lasso, y),
  equicorr = RMSE(y.ko.equi, y),
  randsdp = RMSE(y.ko.rand, y))

##   linear       lasso    equicorr      randsdp
## 50953.33    48708.15    49446.00     42869.85
```

The results highlight the superiority of the SDP-knockoff selected variables, which have a considerable edge over the other methods. Interestingly, the equicorrelated knockoffs do not show an increase in performance over a basic LASSO regression.